COMPLAINT-TRIAGE

Industrialisation

La base et l’image de l’application testées à chaque modification

Nathan Mang
Octobre 2026
Rapport de fin de version

Synthèse

Jusqu'à la v0.4, les tests d'intégration ne tournaient que sur le poste de développement, où PostgreSQL est installé, et l'application n'existait pas sous forme d'image. Cette version, volontairement légère, fait tourner les deux dans la CI de GitHub à chaque PR, sans rien installer sur le poste et sans aucun coût. La CI compte désormais trois tâches en parallèle, toutes vertes dès le premier lancement : les contrôles et les tests unitaires avec les scénarios de sûreté (66 secondes), une base PostgreSQL jetable avec les tests d'intégration (37 secondes) et l'image Docker de l'application (63 secondes).

Table des matières

Synthèse 1

1 Objectif et choix 2

2 La base dans la CI 2

3 L'image Docker 2

4 Bilan 3

1. Objectif et choix

1.1 Une version légère, par choix

Le cadrage prévoyait pour la v0.5 Docker et une chaîne de déploiement complète. Deux contraintes ont conduit à une version plus légère. Le poste de développement n'a que 8 Go de mémoire, et Docker Desktop en occupe à lui seul plusieurs. Et le projet reste sur une règle de zéro dépense : un déploiement sur Google Cloud exige d'y lier un moyen de paiement. Le but de la version est donc ciblé : que la base et l'image de l'application soient testées automatiquement, dans la CI de GitHub, gratuite pour un dépôt public.

1.2 Ce qui est fait, ce qui est reporté

Fait	Reporté, ou écarté
PostgreSQL jetable dans la CI, mis en place par le script du projet ; tests d'intégration à chaque PR	Docker sur le poste : inutile tant que la CI construit et teste l'image
Image Docker de l'application, construite et vérifiée à chaque PR	Publication de l'image dans un registre : utile seulement avec un hébergement
Versions exactes de Python, uv et PostgreSQL	Déploiement sur Google Cloud : remplacé en v1.0 par une démo publique gratuite, sans moyen de paiement

Table 1 – Périmètre de la v0.5

2. La base dans la CI

À chaque PR, GitHub démarre un conteneur PostgreSQL 18.6, la même version que sur le poste, et attend qu'il réponde. Le script de mise en place du projet crée ensuite les comptes, le schéma et les droits, exactement comme sur le poste. C'est aussi un test du script lui-même, sur une machine vierge. Sur le poste, le mot de passe du super-utilisateur reste demandé en saisie masquée ; en CI, il vient d'une variable d'environnement. La base est détruite à la fin de la tâche, et son mot de passe n'est donc pas un secret.

Les dix-huit tests d'intégration tournent alors : les droits de chaque compte, vérifiés en tentant les opérations interdites, et les outils de l'agent sur une banque fictive. Un piège demandait une précaution. Sans base, ces tests sont sautés, et un test sauté ne fait pas échouer la CI : une configuration défaillante passerait donc au vert sans rien avoir testé. Dans cette tâche, une variable `REQUIRE_DATABASE` empêche ce saut ; un test privé de base échoue.

3. L'image Docker

L'image contient le paquet, les scripts et les jeux d'évaluation. Ses choix suivent quatre principes :

— **des versions exactes** : Python 3.12.15, la plus récente au moment de la version, et uv 0.13.1,

comme sur le poste et dans la CI ; toutes deux ont été vérifiées sur les registres avant d'être figées ;

— **les dépendances du fichier** `uv.lock`, installées dans une couche à part, gardée en cache tant que ce fichier ne change pas ;

— **une exécution sans root**, par un utilisateur dédié ;

— **rien de privé** : un fichier `.dockerignore` écarte le fichier `.env`, les données, les environnements, les documents et les tests. Les données se montent au lancement, les secrets passent par les variables d'environnement.

La CI construit l'image, puis vérifie qu'elle démarre, qu'elle importe l'agent et le RAG, qu'elle ne tourne pas en root, et qu'elle ne contient ni fichier `.env`, ni données, ni tests.

4. Bilan

Tâche	Contenu	Durée
Lint, types et tests	ruff, mypy, 205 tests unitaires, 17 scénarios de sûreté	66 s
Base et tests d'intégration	PostgreSQL 18.6, script de mise en place, 18 tests	37 s
Image Docker	Construction et quatre vérifications	63 s

Table 2 – La CI à la fin de la v0.5, premier lancement

Chaque modification du code passe désormais par les mêmes contrôles qu'un projet d'équipe : qualité, sûreté de l'agent, droits de la base, et image de l'application. La v1.0 utilisera cette image pour une démo publique gratuite.